

for msrit qlab primer on 9826

sarmanarasimha

9th August 2026 for Msrit FDP starting 10th August

Qiskit in Google colab

prior to import of commands,install the following.
begin with '!' for each pip install command

```
!pip install qiskit
!pip install qiskit-ibm-runtime
!pip install pylatexenc
!pip install qiskit-aer
!pip install qiskit-algorithms
!pip install qiskit-nature
!pip install qiskit -machine-learning
```

After this , as per context and purpose / requirement— import libraries starting with 'from qiskit.

simulator access

- simulators can be accessed— through / via

```
from qiskit.providers.basic_provider import basic_simulator
```

context by context , case by case, simulator imports differ. so look at the code and proceed respectively. mostly, for simple cases,

```
simulator =AerSimulator(method= ' ')
```

qiskit lightweight more common packages installation

```
!pip install qiskit qiskit-aer matplotlib -q
```

```

# library imports
from qiskit import QuantumCircuit,transpile
from qiskit_aer import Aer Simulator #(simulator import)
from qiskit.visualization import plot_histogram #(qml not requires mostly)
import matplotlib.pyplot as plt #(direct 'mpl' also Ok if taken care)

```

for simple quantum computing only, not QML

previous section and this section almost same, just for clarity only on google colab, one can start with:

```

# simplest package
!pip install qiskit-aer matplotlib pylatexenc -q
#modules
from qiskit import QuantumCircuit,transpile
from qiskit_aer import Aer Simulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt

```

Deutsch-Jozsa using Google's colab

```

!pip install qiskit-aer matplotlib -q
#imports
from qiskit import QuantumCircuit,transpile
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
#D-J oracle
def deutsch_jozsa oracle(circuit,n,oracle_type='balanced'):
    """oracle creation for constant or balanced function"""
    if oracle_type=='balanced':
        circuit.cx(0,n) #output flipping for 1 input qubit(balanced)
    else:
        pass # qubit unchanged(constant)

```

D-J Algorithm

```

def deutsch_jozsa_algorithm(n_qubits=3,oracle_type='balanced'):
    qc=QuantumCircuit(n_qubits+1,n_qubits)
    # here 5 steps are there
    #1--initializing output qubit ket(1)
    qc.x(n_qubits)
    qc.h(n_qubits)
    #2--Hadamard on input qubits

```

```

    for i in range(n_qubits):
        qc.h(i)
    #3--oracle application
    deutsch_jozsa_oracle(qc,n_qubits,oracle_type)
    #4--Hadamard 2nd time
    for i in range(n_qubits):
        qc.h(i)
    #5th--input measurement
    qc.measure(range(n_qubits),range(n_qubits))
    return qc
# D-J run
n_qubits=3
oracle_type='balanced'#whether constant / balanced?
qc=deutsch_jozsa_algorithm(n_qubits,oracle_type)
# using AerSimulator
simulator=AerSimulator()
compiled_circuit=transpile(qc,simulator)
job=simulator.run(compiled_circuit,shots=1024)
result=job.result()
counts=result.get_counts()
# 4 outputs are there
#output 1(Text Circuit)
print("\nDeutsch-Jozsa Quantum Circuit:")
print(qc.draw('text'))
#output 2 (Histogram)
plt.figure(figsize=(6,4))
plot_histogram(counts)
plt.title("Deutsch-Jozsa output distribution({oracle_type.capitalize()}Function)")
plt.show()
#3rd output with XAI inspired Amplitude graph
keys=list(counts.keys())
values=[counts[k]for k in keys]
plt.figure(figsize=(6,4))
plt.bar(keys,values,color='purple',alpha=0.7)
plt.xlabel('Measurement Outcomes')
plt.ylabel('Frequency')
plt.grid(True,linestyle='--',alpha=0.5)
plt.show()
#4th output--interpretation finally,
if '000' in counts:
    print("Function is CONstant(all outcomes are zero.)")
else:
    print("Function is BALanced(Non-zero outcomes detected.)")

```

Grover's using Google colab

```
!pip install qiskit-aer matplotlib -q
#libraries
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit visualization import plot_histogram
import matplotlib.pyplot as plt
#Grover circuit
grover_circuit=QuantumCircuit(2,2)#2 qubit
grover_circuit.h([0,1])#Hadamard
#target marking
grover_circuit.cz(0,1)#oracle to mark ket{1}
#amplitude amplification(diffuser)
grover_circuit.h([0,1])
grover_circuit.x([0,1])
grover_circuit.h(1)
grover_circuit.cx(0,1)
grover_circuit.x([0,1])
grover_circuit.h([0,1])
# to measure
grover_circuit.measure([0,1],[0,1])
simulator=AerSimulator()# simulation run
job=simulator.run(grover_circuit,shots=1024)
result=job.result()
counts=result.get_counts()
#output display
print("Measurement Results:",counts)
#probability histogram
plt.figure(figsize=(6,4))
plot_histogram(counts)
plt.title("Grocer's output distribution",fontsize=14)
plt.show()
```

w.r.to VQE, imports and installations–Google colab

```
!pip install qiskit matplotlib numpy -q
# dataset is implicit
#packages
!pip install qiskit matplotlib numpy -q
#libraries
import numpy as np
```

```

import matplotlib.pyplot as plt
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector, pauli
from scipy.optimize import minimize
#code, optimizer

```

w.r.to QAOA,imports,installation and packages only

```

!pip install pennylane networkx matplotlib numpy -q
# no need of any additional / external dataset
#libraries
import pennylane as qml
from pennylane import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

#then QAOA code

```

imports w.r.to Iris data set classification–(Google colab)

```

!pip install pennylane matplotlib scikit-learn -q #package
#libraries
import pennylane as qml
from pennylane import numpy as np
from sklearn.datasets import load_iris
from sklearn_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# code follows

```